BOOX STORE

The Little Cart Book

A friendly picture-book-style tour of how the shopping basket works

YOUR CART

A cozy basket for books

[1 book] [2 books] [3 books]

Made for curious readers and future checkout adventures.

1. What is a cart?

Imagine walking into a bookshop with a little basket. You see a book you like, so you put it in the basket. The cart on this website is that basket, only it lives on the screen.

When you click **Add to cart**, the cart remembers that book. Add the same book again and its number goes up. The number beside **Cart** in the top menu tells you how many books you have picked altogether.

In your cart you can:

- Use the plus button to add another copy.
- Use the minus button to take one copy out. Taking the last one out removes that book.
- Choose **Remove** to take one book kind out of the basket.
- Choose **Empty cart** to take everything out.
- Look at the subtotal to see the cost of the books currently in the basket.

The basket is shared by the book cards, the menu, and the cart section. So if you add a book on one part of the page, the other parts know about it too.

2. How the book travels

1	You press Add to cart on a book card.
2	The shared cart remembers the book ID and the number of copies.
3	The menu count changes, and the cart shows the book and its current catalogue price.
4	When you change the number, remove a book, or empty the cart, the cart saves the new list.

3. The little helpers behind the screen

The cart is built from a few small helpers. Think of them like the people in a bookshop: one keeps the list, one puts books in, and one shows the basket.

File	Its job
frontend/src/context/CartContext.jsx	Keeps the shared list and the add, change, remove, and empty actions. Saves the list in this browser.
frontend/src/context/cart-context.js	Gives other parts of the page a safe way to ask for the shared cart.
frontend/src/components/BookCard.jsx	Shows each book and its Add to cart button.
frontend/src/components/CartSection.jsx	Shows cart books, quantities, remove buttons, and the subtotal.
frontend/src/components/QuantityStepper.jsx	Shows the plus and minus buttons for a cart book.
frontend/src/components/Header.jsx	Shows the changing book count and a link to the cart.
frontend/src/App.jsx	Loads the book list once and wraps the page so its parts can share the cart.
frontend/src/components/BookGrid.jsx	Shows the books it receives, with the existing category filters and pages.

The book list still comes from the store server. The cart keeps only each book ID and its quantity, not a copied price. When the cart draws the bill, it looks up the current book price from the catalogue.

4. How does it remember?

The cart saves a tiny note in the browser storage on this device. The note says which book IDs are in the basket and how many of each. It does not save a credit-card number, and this cart does not send anything to a payment company.

That means the books can still be there after you refresh the page or come back using the same browser on the same device. The note is not an online account, though. It does not follow you to another phone or computer, and it is not synchronized to your account.

If an old saved book is no longer in the catalogue, the cart tells you and lets you remove it. If this browser cannot read or save its little note, the page shows a message instead of pretending everything worked.

A tiny money example

Book	Copies	Cost
A storybook	2	$14.99 each
Subtotal	**2**	**$29.98**

The page changes prices into whole cents before adding them. That helps avoid tiny decimal rounding hiccups. The subtotal is just the books; shipping and payment are not included yet.

5. Important: the cashier is not here yet

Right now the cart is a basket, not a checkout. It does not take payment, make an order, reserve books, or calculate delivery or tax. The subtotal is a helpful preview, not a final bill.

Why do we need a server cashier later?

A browser is like a note in your pocket: you can change what it says. So when we add Stripe, the server must not trust the browser to tell it the price. The server will check the book IDs and quantities against its own catalogue and calculate the real total.

- The server checks that the shopper is signed in and the chosen books exist.
- The server looks up the real prices and creates a pending order.
- The server asks Stripe to open a secure checkout page.
- Stripe sends a signed message to the server when payment succeeds.
- Only after checking that message should the server mark the order paid.

The success page alone must never say that payment is complete. A customer can return to a page without actually paying; the server must wait for Stripe to confirm.

6. The whole story in one breath

Pick a book, put it in the basket, adjust the copies, and check the subtotal. Your browser remembers the basket on this device. When we build the next checkout chapter, the server and Stripe will do the careful payment work.

The end of the cart chapter - the next chapter can be checkout.